云计算环境下软件微服务架构的智能弹性伸缩算法

滕立飞

（中通服咨询设计研究院有限公司，江苏 南京 210019）

摘　要：在云计算环境下，微服务系统面临负载波动剧烈的挑战，而传统基于阈值的扩缩容方法存在响应滞后与资源利用效率低的问题。为此，文章提出一种融合长短期记忆（Long Short-Term Memory，LSTM）网络与强化学习（Reinforcement Learning，RL）的智能弹性伸缩算法。该算法通过构建服务调用链监控体系实现负载精准预测，并采用深度强化学习（Deep Reinforcement Learning，DRL）动态决策资源调配策略。实验表明，在模拟电商秒杀等高并发场景中，该算法较传统方法的响应时间降低了约 35%，资源利用率提升至约 75%，且能提前约 20 秒预判流量峰值，显著优化了服务质量与资源成本之间的平衡。

关键词：微服务架构；自动扩缩容；负载预测；RL；资源调度

中图分类号：TP183　　**文献标识码**：A　　**文章编号**：1003-9767(2026)22-236-03

Intelligent Elastic Scaling Algorithm for Software Microservices Architecture in Cloud Computing Environments

TENG Lifei

(China Information Consulting & Designing Institute Co., Ltd., Nanjing Jiangsu 210019, China)

Abstract: In the cloud computing environment, microservice systems face the challenge of severe load fluctuations, while traditional threshold based scaling methods suffer from response lag and low resource utilization efficiency. Therefore, the article proposes an intelligent elastic scaling algorithm that integrates long short term memory (LSTM) network and reinforcement learning (RL). The algorithm realizes accurate load prediction by constructing a service call chain monitoring system, and uses deep reinforcement learning (DRL) to dynamically decide resource allocation strategies. Experiments have shown that in high concurrency scenarios such as simulated e-commerce flash sales, this algorithm reduces response time by about 35% compared to traditional methods, improves resource utilization to about 75%, and can predict traffic peaks about 20 seconds in advance, significantly optimizing the balance between service quality and resource costs.

Keywords: microservices architecture; auto-scaling; load prediction; RL; resource scheduling

0　引　言

随着企业数字化转型加速，微服务架构凭借其敏捷性和可扩展性成为云原生应用的核心[1]。然而，微服务实例的动态资源需求与云环境不确定性之间存在显著矛盾。传统基于静态阈值的扩缩容机制难以应对突发流量冲击，常导致资源冗余或服务性能下降[2]。

基于此，本文设计了一种智能弹性伸缩算法。其创新点在于将时序预测与决策优化深度融合，利用注意力机制增强的 LSTM 提升负载预测精度，并通过 DRL 模型自适应生成扩缩容策略。

收稿日期：2026-04-28

作者简介：滕立飞，男，本科，工程师。研究方向：电子专业、信息化、软件、AI、数字化。

中国知网 https://www.cnki.net

1 微服务调度现状与核心挑战

早期单体架构随业务膨胀易引发连锁故障，而微服务将系统拆分为独立单元，支持异构技术栈与容器化部署。这一演进虽提升了开发效率，但也带来了管理复杂度的指数级增长：调用链路深达数层，局部抖动易传导至全系统。技术架构从单机、集群、服务化治理逐步发展至云原生阶段，但云环境资源调度仍面临三大难题：一是编程语言启动时间差异大，若按统一规则扩容，会出现“新实例未就绪而旧实例已过载”的问题；二是服务依赖引发连锁反应，上游未及时处理会导致下游积压；三是突发流量增长远超传统阈值响应速度——传统策略数据采集决策需分钟级，而流量可在 10 s 内增长数十倍，导致以中央处理器（Central Processing Unit，CPU）利用率大于 80%作为扩容触发条件的传统策略失效。因此，亟须制订快速精准的扩缩容决策[3]。

2 算法架构设计

2.1 系统整体框架

本算法采用监控、预测、决策三层结构。底层收集各类运行数据，包括服务器层面的 CPU 与内存占用、服务层面的请求数量与响应时间，以及服务间调用关系；中间层利用神经网络分析历史数据，预测未来数分钟的负载变化趋势；顶层基于 RL 模型，通过试错学习最优策略，自主决策扩缩容时机与规模。各模块数据流向清晰，原始数据经清洗预处理后输入预测模块，预测结果与当前资源状态共同输入决策模块，最终输出具体扩缩容指令。整个流程自动化运行，无需人工干预并形成闭环反馈[4]。

2.2 关键参数定义

为使算法有效运行，算法需定义三个核心要素：状态描述系统当前情况，包括实例数量、预测负载、资源成本和响应是否超时等；动作指可选的扩缩容操作，如减少 2 个实例、保持不变、增加 1 个或 3 个实例，幅度明确；反馈采用评分机制，预测准确且资源利用率高时给予高分，出现资源浪费或服务超时则给予负分[5]。通过持续试错，算法学会在不同状态下选择得分最高的操作，从而形成最优策略。该评分机制如公式（1）所示：

$$R = -0.4 \times |E_p| - 0.3 \times U_w - 0.3 \times N_t \quad (1)$$

其中，R 为奖励值，E_p 为预测误差，U_w 为资源浪费率，N_t 为服务超时次数。

该评分机制基于多目标优化理论，通过加权求和将多目标转化为单一目标函数，最小化预测误差、资源浪费与服务超时，引导 RL 智能体学习最优策略，其中，E_p 项权重最高，表明精准预测是首要目标；U_w 与 N_t 权重相等，体现资源效率与服务质量同等重要。在实际应用中，可根据业务需求灵活调整三项权重，如成本敏感型业务可适当提高资源浪费率权重，用户体验优先型业务可提升服务超时次数权重[6]。

3 核心算法实现

3.1 负载预测模型构建

预测模块采用 LSTM 处理时序数据，学习历史负载演变规律。模型输入过去一段时间的多维运行指标（如请求量、响应时间、错误率），在编码器-解码器框架中集成加性注意力机制，并通过对编码器各时间步的隐藏状态加权求和，动态生成上下文向量。这使得解码器在预测时能重点关注历史中的关键时段，从而有效捕捉突发流量等非平稳模式[7]。训练目标为最小化预测误差，损失函数由两部分构成，如公式（2）所示：

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \cdot T_c(y, \hat{y}) \quad (2)$$

其中，L 表示损失函数值，n 表示样本数量，y_i 代表第 i 个样本的真实负载值，$\hat{y}_i$ 为模型预测值，λ 为趋势一致性惩罚项的权重系数，$T_c(y, \hat{y})$ 为趋势一致性惩罚项。式中第一项为均方误差，用于衡量预测值与真实值的偏离程度；第二项惩罚预测趋势与实际趋势不一致的情况，通过设置 $\lambda = 0.1$ 在数值精度与趋势一致性之间取得平衡。模型经梯度下降优化后，预测误差稳定在较低水平，为后续决策提供可靠依据。

3.2 智能决策引擎设计

决策模块采用双网络结构，缓解了传统 RL 的过度乐观估计问题。神经网络共三层，神经元数量分别为 256、128、5，使用了 ReLU 激活函数。学习过程平衡探索与利用：初期随机尝试不同动作以充分探索环境，后期则依据高分动作实现收益最大化。探索率初始设为 0.3，并采用指数衰减策略（衰减因子 0.999 995）。根据指数衰减公式，经过约 10 万步训练后，探索率将衰减至 0.01。这一机制使得算法在训练后期能够聚焦于已有策略，从而实现稳定收敛。模型与模拟环境交互 10 万步，逐步掌握不同负载下的最优扩缩容时机。训练完成后，单次决策耗时不足 5 ms，满足了实时响应要求。该设计确保了算法既能充分探索策略空间，又能快速收敛至高性能策略，为实际部署奠定了坚实基础。

4 仿真实验与性能分析

4.1 实验环境搭建与实验流程

本文定义“过度扩缩容次数”：在任意评估周期内，因预测偏差导致的、超出实际资源需求阈值（本文设定为±20%）

中国知网 https://www.cnki.net

的扩缩容操作次数。该指标用于衡量算法决策的精确性与稳定性。为验证算法有效性，在实验室环境中搭建了模拟云平台，选用了开源微服务测试程序 DeathStarBench 中的社交网络应用，并部署在 100 台虚拟服务器上。

实验设计了三种典型场景：一是平稳增长型，模拟日常业务缓慢爬升，持续 4 h；二是突发激增型，模拟电商秒杀场景，要求在 10 min 内请求量增长 50 倍；三是随机波动型，通过不规则注入流量，测试算法的抗干扰能力[8]。对比方案选取业界通用的 Kubernetes 自动扩缩容 HPA（Horizontal Pod Autoscaler）与一种改进预测式方案。为确保结果可靠性，每种场景重复运行 5 次取平均值。该实验设置既覆盖了真实业务中的常见模式，又通过重复实验降低了随机误差的影响，为后续性能分析提供了可信基础[9]。

4.2　实验结果分析

测试结果显示，在突发激增场景（模拟电商秒杀及春晚红包流量）下，本文算法的平均响应时间约为 90 ms，相比传统阈值方案（Kubernetes HPA）缩短约 35%；资源利用率维持在 75%左右，比对照组提高约 15 个百分点；过度扩缩容次数减少约 40%。这些数据印证了算法在性能与效率方面的双重优化。特别是算法能提前约 20 s 预判流量峰值并预热扩容，而传统方案多在流量到达后才触发扩容，易导致服务短暂卡顿。

为直观对比三种算法在突发负载下的动态响应差异，绘制了如图 1 所示的平均响应时间时序折线图。数据取自每秒聚合均值，涵盖流量激增前期、峰值爆发和恢复阶段。

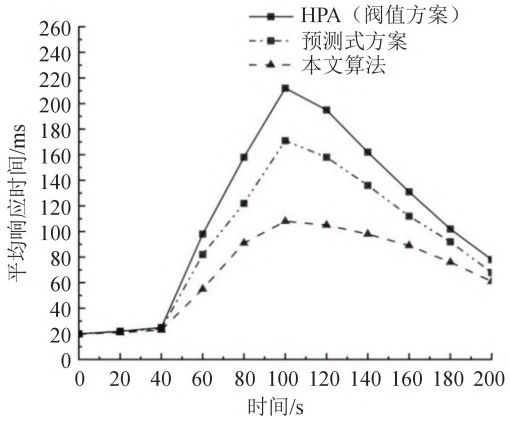


图 1　不同弹性伸缩算法在突发流量下的响应时间对比

从图 1 可见，在约 60～200 s 的高负载窗口中，传统 HPA 响应时间快速攀升，峰值超过 210 ms 且回落滞后；预测式方案峰值约 170 ms；而本文算法在流量涌起前（约 40～50 s）已平滑预备，峰值被抑制在 108 ms 以内，波动幅度最小。定量统计表明，在激增时段，本文算法的平均响应时间为 89.8 ms，较 HPA（141.2 ms）优化约 36.4%，较预测式方案（118.9 ms）优化约 24.5%，且曲线恢复更快、更平稳。这种预判式调节有效避免了资源空转与服务超时，与奖励函数中“预测误差权重最高”的目标一致。综合来看，该智能算法既保障了服务质量，又降低了资源成本，验证了其在真实业务场景中的实用价值与推广潜力。

5　结　语

本研究验证了智能算法在微服务弹性伸缩中的有效性与实用性，通过 LSTM 与 RL 的协同优化，实现了资源效率与服务质量的同步提升，为云环境下的微服务资源管理提供了新思路。未来研究工作将聚焦于异构计算资源的协同调度机制，探索图形处理器等专用硬件的集成方案，同时深入分析多租户环境下的资源隔离需求，以进一步增强算法在复杂生产场景中的适应性与普适性，推动云计算资源管理的智能化发展。

参考文献

[1] 高程. 云计算环境下的网络安全技术应用研究[J]. 软件，2025，46(11)：152-154.

[2] 靳宇珍. 云计算环境下计算机软件开发框架与性能优化[J]. 信息记录材料，2025，26(9)：164-166.

[3] 耿张燕. 基于云计算的软件架构设计与优化分析[J]. 电子技术，2025，54(2)：100-101.

[4] 宋玉，孙贺，王家兴，等. 云计算环境下的拟态 IAM 系统设计实现方法[J]. 计算机工程与设计，2024，45(5)：1297-1304.

[5] 陈妍，韦湘，陆臻. 云计算环境下网络安全产品检测方法研究[J]. 信息网络安全，2022，22(12)：1-6.

[6] 陈志兰. 云计算环境下的跨平台软件整合分析[J]. 电子技术，2024，53(6)：184-185.

[7] 张翔越，许伟，高峰，等. 基于云架构的调度集中系统软件测试方案研究[J]. 铁道通信信号，2026，62(3)：1-7.

[8] 韩世平，李林，陈杰，等. 基于微服务架构的多源异构数据实时处理平台设计[J]. 国外电子测量技术，2025，44(11)：251-256.

[9] 李光文，左妍，刘楠，等. 基于微服务架构的专家遴选系统设计与实现[J]. 科技创新与生产力，2025，46(9)：142-148.

中国知网 https://www.cnki.net